I1 · Les dates

~2.5 h

À la fin de cette séance vous saurez

- convertir du texte et des dates-heures Excel en vraies **dates** avec `as.Date()` et son `format` ;
- extraire l'année, le mois, le jour de la semaine ; calculer des **durées** entre deux dates ;
- créer des **saisons** et des périodes à partir des dates ;
- utiliser **lubridate** pour faire la même chose avec un code plus court ;
- calculer le **taux de croissance** des algues de chaque ligne.

> **i** Partez de votre script I0, ou du fichier de contrôle : `lines <- read.csv("data/checkpoints/I0_lines.csv")` . Avec le fichier de contrôle, `planting_date` est du texte `"2024-01-08"` : convertissez-le d'abord avec `as.Date()` .

⇩ checkpoint: I0_lines.csv

Pourquoi les dates demandent de l'attention

Pour R, `"22/02/2024"` n'est que du texte : il ne peut pas calculer que 45 jours séparent le semis de la récolte, ni trier correctement les dates (`"02/03/2024"` passerait avant `"22/02/2024"` dans l'ordre alphabétique). Un objet **Date** stocke le nombre de jours depuis le 1er janvier 1970, ce qui rend tous les calculs possibles.

```
d <- as.Date("2024-01-08")      # the international format yyyy-mm-dd is read dir
ectly
class(d)
```

```
#> [1] "Date"
```

```
as.numeric(d)                   # days since 1970-01-01
```

```
#> [1] 19730
```

```
d + 45                          # 45 days later
```

```
#> [1] "2024-02-22"
```

Du texte à la date : as.Date() et format

Quand le texte n'est pas en `aaaa-mm-jj`, indiquez à R comment il est écrit avec des **codes de format** :

Code	Signification	Exemple
`%d`	jour (01–31)	`08`
`%m`	numéro du mois (01–12)	`03`
`%b` / `%B`	nom du mois, court / complet (dans la langue de l'ordinateur)	`mars` / `mars`
`%y` / `%Y`	année, 2 / 4 chiffres	`24` / `2024`
`%A`	jour de la semaine	`lundi`

```
as.Date("22/02/2024", format = "%d/%m/%Y")
```

```
#> [1] "2024-02-22"
```

```
as.Date("02-22-24", format = "%m-%d-%y")
```

```
#> [1] "2024-02-22"
```

```
lines$harvest_date[1:3]
```

```
#> [1] "22/02/2024" "20/02/2024" "26/02/2024"
```

```
lines$harvest_date <- as.Date(lines$harvest_date, format = "%d/%m/%Y")
lines$harvest_date[1:3]
```

```
#> [1] "2024-02-22" "2024-02-20" "2024-02-26"
```

! Si le format ne correspond pas au texte, `as.Date()` renvoie `NA` **sans erreur**. Vérifiez toujours : `sum(is.na(x))` avant et après conversion doivent être égaux.

Dates Excel lues comme des nombres (ex. `45299`) : `as.Date(45299, origin = "1899-12-30")` . **Dates-heures** (`POSIXct` , de readxl) : `as.Date(x)` .

Durées, parties d'une date, séquences

```
lines$days <- as.numeric(lines$harvest_date - lines$planting_date)   # difference in days
summary(lines$days)
```

```
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
#>   40.00   42.00   45.00   45.02   48.00   50.00
```

```
format(lines$planting_date[1:3], "%Y")                  # year as text
```

```
#> [1] "2024" "2024" "2024"
```

```
as.integer(format(lines$planting_date[1:3], "%m"))      # month as a number
```

```
#> [1] 1 1 1
```

```
format(as.Date("2024-01-08"), "%d %B %Y")               # nice label for a report
```

```
#> [1] "08 January 2024"
```

```
difftime(as.Date("2025-12-31"), as.Date("2024-01-01"), units = "weeks")
```

```
#> Time difference of 104.2857 weeks
```

```
seq(as.Date("2024-01-01"), by = "month", length.out = 4)   # monthly sampling calendar
```

```
#> [1] "2024-01-01" "2024-02-01" "2024-03-01" "2024-04-01"
```

Saisons et périodes

Dans cette région, la **saison des pluies** va de novembre à avril, la **saison sèche** de mai à octobre. Une saison est une catégorie construite à partir du numéro du mois avec `%in%` et `ifelse()` (`ifelse()` sera étudié en détail en séance I5).

```
lines$year   <- as.integer(format(lines$planting_date, "%Y"))
lines$month  <- as.integer(format(lines$planting_date, "%m"))
lines$season <- ifelse(lines$month %in% c(11, 12, 1, 2, 3, 4), "wet", "dry")
table(lines$year, lines$season)
```

```
#>
#>        dry wet
#>   2024 692 696
#>   2025 652 535
```

`cut()` regroupe les dates en périodes, par exemple en trimestres :

```
head(cut(lines$planting_date, breaks = "quarter"))
```

```
#> [1] 2024-01-01 2024-01-01 2024-01-01 2024-01-01 2024-01-01 2024-01-01
#> 8 Levels: 2024-01-01 2024-04-01 2024-07-01 2024-10-01 ... 2025-10-01
```

La même chose avec lubridate

lubridate (tidyverse) propose des fonctions nommées d'après l'ordre des éléments : `dmy()` pour jour-mois-année, `ymd()` , `mdy()` … Pas de codes de format à retenir. Et `year()` , `month()` , `wday()` extraient directement les éléments sous forme de nombres.

R BASE

```
as.Date("22/02/2024", format = "%d/%m/%Y")
```

```
#> [1] "2024-02-22"
```

```
as.integer(format(as.Date("2024-02-22"), "%m"))
```

LUBRIDATE

```
library(lubridate)
dmy("22/02/2024")
```

```
#> [1] "2024-02-22"
```

```
month(ymd("2024-02-22"))
```

by Aroo Falinirina

```
#> [1] 2
```

```
#> [1] 2
```

```
as.Date("2024-02-22") + 30
```

```
ymd("2024-02-22") + days(30)
```

```
#> [1] "2024-03-23"
```

```
#> [1] "2024-03-23"
```

```
dmy(c("22/02/2024", "3-4-2024", "05.06.2024"))   # tolerant to separators
```

```
#> [1] "2024-02-22" "2024-04-03" "2024-06-05"
```

```
floor_date(ymd("2024-02-22"), "month")          # first day of the month
```

```
#> [1] "2024-02-01"
```

Taux de croissance de chaque ligne

~ Producteurs et chercheurs comparent les lignes avec le **taux de croissance spécifique** (SGR), en % par jour :

SGR = 100 × ln(poids final / poids initial) / nombre de jours

Il tient compte des poids initiaux et des durées de cycle différents. Des valeurs de 3 à 5 %/jour sont habituelles pour *Kappaphycus*.

```
i <- which(lines$final_weight_kg > 200)                 # weights typed in grams (I0)
lines$final_weight_kg[i] <- lines$final_weight_kg[i] / 1000
lines$sgr <- 100 * log(lines$final_weight_kg / lines$initial_weight_kg) / lines$days
summary(lines$sgr)
```

```
#>    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's
#>   1.405   2.967   3.445   3.439   3.882   5.456     258
```

```
round(tapply(lines$sgr, lines$season, mean, na.rm = TRUE), 2)
```

```
#>  dry  wet
#> 3.56 3.30
```

⇩ checkpoint: I1_lines_dated.csv

Travail d’application et quiz

Travail d’application I1

Script `scripts/i01_dates.R` , avec la feuille `lines` (ou le fichier de contrôle).

1. Convertissez `harvest_date` en date. Quelle est la **première** date de récolte (`aaaa-mm-jj`) ?
2. Ajoutez `days` (récolte − semis). Durée moyenne (2 décimales) ? Durée la plus longue ?
3. Ajoutez `year` , `month` (à partir de la date de semis) et `season` . Combien de lignes ont été semées en 2025 ?
4. Quel mois (numéro 1–12) compte le plus de lignes semées ?
5. Combien de lignes ont été semées en saison des pluies ?
6. Corrigez les poids en grammes, puis calculez `sgr` . SGR moyen de toutes les lignes (3 décimales, sans NA) ? SGR moyen en saison sèche (3 décimales) ?
7. Combien de jours entre le 1er janvier 2024 et le 31 décembre 2025 ?

> Question 4 : `table(lines$month)` puis `which.max()` ; la réponse est le **nom** de la case, c’est-à-dire le numéro du mois. Question 6 : le logarithme népérien est `log()` dans R.

Quiz : répondez en ligne sur le site du cours (votre score y est enregistré).